PEP 593: Flexible function and variable annotations

Status: Final · Created: 26-Apr-2019 · Public domain / CC0

Abstract

This PEP introduces a mechanism to extend the type annotations from PEP 484 with arbitrary metadata.

Motivation

484 provides a standard semantic for the annotations introduced in 3107. 484 is prescriptive but it is the de facto standard for most of the consumers of annotations; in many statically checked code bases, where type annotations are widely used, they have effectively crowded out any other form of annotation. Some of the use cases for annotations described in 3107 (database mapping, foreign languages bridge) are not currently realistic given the prevalence of type annotations. Furthermore, the standardisation of type annotations rules out advanced features only supported by specific type checkers.

Rationale

This PEP adds an Annotated type to the typing module to decorate existing types with context-specific metadata. Specifically, a type T can be annotated with metadata x via the typehint Annotated[T, x]. This metadata can be used for either static analysis or at runtime. If a library (or tool) encounters a typehint Annotated[T, x] and has no special logic for metadata x, it should ignore it and simply treat the type as T. Unlike the no_type_check functionality that currently exists in the typing module which completely disables typechecking annotations on a function or a class, the Annotated type allows for both static typechecking of T (e.g., via mypy or Pyre, which can safely ignore x) together with runtime access to x within a specific application. The introduction of this type would address a diverse set of use cases of interest to the broader Python community.

This was originally brought up as issue 600 in the typing github and then discussed in Python ideas.

Motivating examples

Combining runtime and static uses of annotations

There's an emerging trend of libraries leveraging the typing annotations at runtime (e.g.: dataclasses); having the ability to extend the typing annotations with external data would be a great boon for those libraries.

Here's an example of how a hypothetical module could leverage annotations to read c structs::

```
UnsignedShort = Annotated[int, struct2.ctype('H')]
SignedChar = Annotated[int, struct2.ctype('b')]
class Student(struct2.Packed):
    # mypy typechecks 'name' field as 'str'
    name: Annotated[str, struct2.ctype("<10s")]
    serialnum: UnsignedShort
    school: SignedChar
# 'unpack' only uses the metadata within the type annotations
Student.unpack(record)
# Student(name=b'raymond   ', serialnum=4658, school=264)
```

Lowering barriers to developing new typing constructs

Typically when adding a new type, a developer need to upstream that type to the typing module and change mypy, PyCharm, Pyre, pytype, etc... This is particularly important when working on

open-source code that makes use of these types, seeing as the code would not be immediately transportable to other developers' tools without additional logic. As a result, there is a high cost to developing and trying out new types in a codebase. Ideally, authors should be able to introduce new types in a manner that allows for graceful degradation (e.g.: when clients do not have a custom mypy plugin), which would lower the barrier to development and ensure some degree of backward compatibility.

For example, suppose that an author wanted to add support for tagged unions to Python. One way to accomplish would be to annotate TypedDict in Python such that only one field is allowed to be set::

```
Currency = Annotated[
    TypedDict('Currency', {'dollars': float, 'pounds': float}, total=False),
    TaggedUnion,
]
```

This is a somewhat cumbersome syntax but it allows us to iterate on this proof-of-concept and have people with type checkers (or other tools) that don't yet support this feature work in a codebase with tagged unions. The author could easily test this proposal and iron out the kinks before trying to upstream tagged union to typing, mypy, etc. Moreover, tools that do not have support for parsing the TaggedUnion annotation would still be able to treat Currency as a TypedDict, which is still a close approximation (slightly less strict).

Specification

Syntax

Annotated is parameterized with a type and an arbitrary list of Python values that represent the annotations. Here are the specific details of the syntax:

* The first argument to Annotated must be a valid type

* Multiple type annotations are supported (Annotated supports variadic arguments)::

```
Annotated[int, ValueRange(3, 10), ctype("char")]
```

* Annotated must be called with at least two arguments (Annotated[int] is not valid)

* The order of the annotations is preserved and matters for equality checks::

```
Annotated[int, ValueRange(3, 10), ctype("char")] != Annotated[
    int, ctype("char"), ValueRange(3, 10)
]
```

* Nested Annotated types are flattened, with metadata ordered starting with the innermost annotation::

```
Annotated[Annotated[int, ValueRange(3, 10)], ctype("char")] == Annotated[
    int, ValueRange(3, 10), ctype("char")
]
```

* Duplicated annotations are not removed::

```
Annotated[int, ValueRange(3, 10)] != Annotated[
    int, ValueRange(3, 10), ValueRange(3, 10)
]
```

* Annotated can be used with nested and generic aliases::

```
Typevar T = ...
Vec = Annotated[List[Tuple[T, T]], MaxLen(10)]
V = Vec[int]
V == Annotated[List[Tuple[int, int]], MaxLen(10)]
```

Consuming annotations

Ultimately, the responsibility of how to interpret the annotations (if at all) is the responsibility of the tool or library encountering the Annotated type. A tool or library encountering an Annotated type can scan through the annotations to determine if they are of interest (e.g., using isinstance()).

Unknown annotations: When a tool or a library does not support annotations or encounters an unknown annotation it should just ignore it and treat annotated type as the underlying type. For example, when encountering an annotation that is not an instance of struct2.ctype to the annotations for name (e.g., Annotated[str, 'foo', struct2.ctype("<10s")]), the unpack method should ignore it.

Namespacing annotations: Namespaces are not needed for annotations since the class used by the annotations acts as a namespace.

Multiple annotations: It's up to the tool consuming the annotations to decide whether the client is allowed to have several annotations on one type and how to merge those annotations.

Since the Annotated type allows you to put several annotations of the same (or different) type(s) on any node, the tools or libraries consuming those annotations are in charge of dealing with potential duplicates. For example, if you are doing value range analysis you might allow this::

```
T1 = Annotated[int, ValueRange(-10, 5)]
T2 = Annotated[T1, ValueRange(-20, 3)]
```

Flattening nested annotations, this translates to::

```
T2 = Annotated[int, ValueRange(-10, 5), ValueRange(-20, 3)]
```

Interaction with get_type_hints()

typing.get_type_hints() will take a new argument include_extras that defaults to False to preserve backward compatibility. When include_extras is False, the extra annotations will be stripped out of the returned value. Otherwise, the annotations will be returned unchanged::

```
@struct2.packed
class Student(NamedTuple):
    name: Annotated[str, struct.ctype("<10s")]
get_type_hints(Student) == {'name': str}
get_type_hints(Student, include_extras=False) == {'name': str}
get_type_hints(Student, include_extras=True) == {
    'name': Annotated[str, struct.ctype("<10s")]
}
```

Aliases & Concerns over verbosity

Writing typing.Annotated everywhere can be quite verbose; fortunately, the ability to alias annotations means that in practice we don't expect clients to have to write lots of boilerplate code::

```
T = TypeVar('T')
Const = Annotated[T, my_annotations.CONST]
class C:
    def const_method(self: Const[List[int]]) -> int:
        ...
```

Rejected ideas

Some of the proposed ideas were rejected from this PEP because they would cause Annotated to not integrate cleanly with the other typing annotations:

* Annotated cannot infer the decorated type. You could imagine that Annotated[..., Immutable] could be used to mark a value as immutable while still inferring its type. Typing does not support using the inferred type anywhere else; it's best to not add this as a special case.

* Using (Type, Ann1, Ann2, ...) instead of Annotated[Type, Ann1, Ann2, ...]. This would cause confusion when annotations appear in nested positions (Callable[[A, B], C] is too similar to Callable[[(A, B)], C]) and would make it impossible for constructors to be passthrough (T(5) == C(5) when C = Annotation[T, Ann]).

This feature was left out to keep the design simple:

* Annotated cannot be called with a single argument. Annotated could support returning the underlying value when called with a single argument (e.g.: Annotated[int] == int). This complicates the specifications and adds little benefit.

Copyright

This document has been placed in the public domain.